```
}
```

#A Inject the URI of the token introspection endpoint.
#B Build up HTTP Basic credentials from the client ID and secret.
#C Throw an exception to disable direct login and logout.

To validate a token, you then need to make a POST request to the introspection endpoint passing the token. You can use the HTTP client library in `java.net.http`, which was added in Java 11 (for earlier versions you can use Apache HttpComponents, https://hc.apache.org/httpcomponents-client-ga/). Because the token is untrusted before the call, you should first validate it to ensure that it conforms to the allowed syntax for access tokens. As you learned in chapter 2, it's important to always validate all inputs, and this is especially important when the input will be included in a call to another system. The standard doesn't specify a maximum size for access tokens, but you should enforce a limit of around 1KB or less, which should be enough for most token formats (if the access token is a JWT it could get quite large). The token should then be URL-encoded to include in the POST body as the `token` parameter. It's important to properly encode parameters when calling another system to prevent an attacker being able to manipulate the content of the request (see section 2.6 of chapter 2). You can also include a `token_type_hint` parameter to indicate that it's an access token, but this is optional.

> **TIP** To avoid making an HTTP call every time a client uses an access token with your API, you can cache the response for a short period of time, indexed by the